

1 Computer System Overview

Main Elements and Registers

Processor	executes instructions and controls system operation.
Main memory	stores currently used programs and data.
I/O modules	move data between the computer and external devices.
System bus	carries data, address, and control signals among modules.

User-visible registers: referenced by machine instructions to reduce memory access. Control/status registers: PC, IR, MAR, MBR, PSW; used by CPU and OS to control execution.

Instruction Actions

- Processor-memory: load/store between CPU and memory.
- Processor-I/O: transfer data between CPU and I/O modules.
- Data processing: arithmetic or logical operations.
- Control: branch, jump, call, return; changes execution order.

Interrupts

Purpose and Types

Interrupt: a mechanism by which I/O, timer, program error, or hardware failure interrupts normal processor execution. Main purpose: improve processor utilization.

Program	divide by zero, overflow, illegal instruction.
Timer	lets the OS regain control periodically.
I/O	device reports completion or readiness.
Hardware failure	power failure, memory parity error.

Processing Sequence

1. Device issues interrupt; CPU finishes current instruction.
2. CPU acknowledges, saves PC and PSW, and loads ISR address.
3. ISR saves remaining state, handles the event, restores state.
4. Old PC and PSW are restored; interrupted program resumes.

PC tells where to resume; PSW records processor state. Polling repeatedly tests device flags; interrupts let the device notify the CPU.

Multiple Interrupts

Sequential: disable interrupts during ISR; new interrupts stay pending. Simple but may delay urgent events.

Nested: assign priorities; a higher-priority interrupt may interrupt a lower-priority ISR.

Memory, Cache, I/O

Memory Hierarchy

Higher levels are faster, smaller, and more expensive per bit; lower levels are larger and slower. Distinguish levels by capacity, access time, cost per bit, and frequency of access.

$$T_{avg} = \sum_i \left(\prod_{j < i} (1 - H_j) \right) T_i$$
$$C_{avg} = \sum C_i S_i / \sum S_i$$

Cache and Locality

Cache: small fast memory between processor and main memory; stores recently or frequently used blocks to reduce average access time.

Temporal locality: a referenced item is likely to be referenced again soon; keep recent items in fast memory.

Spatial locality: nearby addresses are likely to be referenced soon; fetch a block, not only one word.

I/O and DMA

Programmed I/O: CPU waits by polling flags such as FGI/FGO.

Interrupt-driven I/O: IEN enables the device to interrupt when ready.

DMA: transfers data directly between I/O device and memory; DMA gets high memory priority because device delay may cause data loss, while CPU delay usually only slows execution.

Chapter 1 Exam Questions

Review 1.1–1.10

Q: List the four main computer elements.
A: Processor, main memory, I/O modules, system bus.
Q: Define the two register categories.
A: User-visible registers; control and status registers.
Q: What is an interrupt?
A: A mechanism that lets another module interrupt normal CPU execution so an ISR can handle the event.
Q: How are multiple interrupts handled?

A: Sequential processing disables interrupts during ISR; nested processing uses priorities.

Q: What distinguishes memory hierarchy levels?

A: Capacity, access time, cost per bit, and access frequency.

Q: Multiprocessor vs multicore?

A: Multiprocessor has multiple processors; multicore has multiple cores on one chip.

Problem Stems and Answers

1.1 I/O program	Load dev5, add M[940], store dev6: dev6 receives 0005. Expand Fig. 1.4 execution: fetch via MAR/MBR; final M[941] = 0005. 24-bit address field gives 16 MB; PC = 24 bits, IR = 32 bits. 16-bit memory: 128 KiB; 8-bit port number: 256 ports. 8 MHz, 16-bit bus, 4 clocks/bus cycle gives 4 MB/s; wider bus or doubled clock gives 8 MB/s. FGI/FGO polling wastes CPU; IEN enables interrupt-driven I/O. DMA gets memory priority because device delay may lose data; CPU delay mostly slows execution.
1.2 MAR/MBR	9600 bps DMA slows 1 MIPS CPU by about 0.12%. Programmed I/O = 25,000 words/s; DMA = 2.15×10^6 words/s. Spatial: sequential a[i]; temporal: repeated same a[i] in inner loop.
1.3 32-bit instr.	
1.4 16-bit addr.	
1.5 bus rate	
1.6 Teletype I/O	
1.7 DMA priority	
1.8 DMA slowdown	
1.9 I/O rate	
1.10 locality code	
1.11 n-level memory	
1.12 cost/hit	
1.13 cache/disk	
1.14 stack PC	

2.1 OS Objectives and Functions

Definition

Operating system: a program that controls application execution and acts as an interface between applications and computer hardware.

Three Objectives

Convenience	makes the computer easier to use.
Efficiency	uses CPU, memory, I/O, files, storage, and network resources efficiently.
Evolution	permits new functions, tests, fixes, and hardware support without disrupting service.

Two Main Roles

- User/computer interface: hides hardware details and provides usable services.
- Resource manager: allocates processor, memory, I/O devices, files, storage, and network resources.

OS Services

Program development	editors, debuggers, compilers, libraries.
Program execution	loads code/data, initializes files, I/O, and resources.
I/O access	uniform device interface, e.g. read(), write().
File access	file organization, permissions, protection.
System access	login, authorization, resource contention control.
Error handling	detect hardware/software errors; terminate, retry, or report.
Accounting	collect usage/performance data; support billing and tuning.

Interfaces: ISA vs ABI vs API

ISA	CPU instruction set	hardware/software boundary; user ISA vs system ISA.
ABI	binary compatibility	system-call interface, registers, calling convention, binary format.
API	source-level calls	library functions used by programmers, e.g. open(), malloc().

Memory hook: API = coding; ABI = compiled binary execution; ISA = CPU instructions.

Kernel, Control, Evolution

Kernel / Nucleus

Kernel: central OS part, normally resident in main memory, containing the most frequently used OS functions: process, memory, I/O, file, interrupt, and system-call handling.

How the OS Controls the CPU

The OS is also software. It gains control, schedules and allocates resources, gives control to user programs, then regains control through interrupts, system calls, or exceptions.

Why OSs Must Evolve

- Hardware upgrades: new processors/devices require new support.
- New services: users and administrators need new tools and features.
- Fixes: bugs must be repaired without destabilizing old services.

System Calls and Modes

System Calls

System call: controlled entry point for a user program to request OS services such as file I/O, process creation, memory allocation, communication, and protection.

Dual-Mode Operation

User mode	low privilege; applications cannot directly execute privileged instructions.
Kernel mode	high privilege; OS kernel performs protected operations.

Flow: a user program requests a system call/trap; the kernel performs the service and returns to user mode.

PDF Stems and Answers

- R2.1 Stem: three OS-design objectives. A: convenience, efficiency, evolution.
- R2.2 Stem: kernel of an OS. A: memory-resident core with frequent OS functions.
- P2.4 Stem: system-call purpose and dual-mode relation. A: safe service request; user mode traps to kernel mode, then returns.
- P2.5 Stem: OS/390 SRM checks untouched page frames. A: find inactive pages, guide replacement/reallocation, detect memory pressure or thrashing.
- R2.6 Stem: five storage duties. A: isolation, allocation, modular support, protection/access control, long-term storage.

Chapter 2 Fast Recall

- OS = application controller + hardware interface + resource manager.
- OS hides hardware details behind services and interfaces.
- Important services: development, execution, I/O, files, access control, errors, accounting.
- Kernel holds common privileged OS functions in memory.
- System calls are the legal bridge from user mode to kernel mode.
- Evolution needs modular design, clear interfaces, and maintainability.

3 Process Description and Control

Process, Image, PCB

Process: a program in execution; an entity that can be assigned to and executed by a processor.

Process image: program code, data, stack, and process attributes.

PCB: OS record holding enough information to stop and later resume a process as if it had not been interrupted.

ID	PID, parent PID, user ID.
CPU state	registers, PC, PSW, stack pointer.
Control	state, priority, scheduling links, memory pointers, I/O status, accounting.

Instruction trace: sequence of instructions actually executed by a process.

OS tables: memory, I/O, file, and process

tables.

Five-State Model

New	created but not yet admitted.
Ready	can run once assigned a CPU; lacks only CPU.
Running	currently executing on the CPU.
Blocked	waits for an event such as I/O completion.
Exit	terminated or released.

Admit: New → Ready. **Dispatch:** Ready → Running. **Timeout/preempt:** Running → Ready. **Event wait:** Running → Blocked. **Event occurs:** Blocked → Ready. **Release:** Running → Exit.

Ready = waiting for CPU; Blocked = waiting for an event. An event-completed blocked process returns to Ready, not directly to Running.

Suspension and Swapping

Swapping: moving a process between main memory and disk to free memory and improve CPU use.

Suspended: not immediately available for execution; may or may not be waiting for an event.

Ready/Suspend	swapped out; event condition satisfied; can run after swap-in.
Blocked/Suspend	swapped out and still waiting for an event.
Reasons	swapping, OS reason, interactive user request, timing, parent request.
Traits	not executable now; may wait on event; agent-suspended; needs explicit activation.

Queues: use one Ready queue plus per-event Blocked queues; suspended states use Ready/Suspend and per-event Blocked/-Suspend queues.

Control, Modes, Switching

Creation steps: assign PID; allocate process space; initialize PCB; link into ready or suspend queues; create/expand accounting and other tables.

User mode: low privilege; cannot execute privileged instructions. **Kernel/System mode:** high privilege for protected OS operations.

Interrupt: asynchronous external event. **Trap:** exception caused by the current instruction. **Supervisor call:** explicit request for an OS service.

Mode switch: changes privilege mode, possibly same process. **Process switch:** CPU

changes from one process to another.

Process switch work: save old PC, PSW, registers into PCB; update state and queues; choose next process; load next PCB/context.

Chapter 3 Review Stems

R3.1 Trace	Stem: instruction trace. A: executed instruction sequence of a process.
R3.2 Creation	Stem: process creation events. A: new batch job, interactive logon, OS-created service, spawned child.
R3.3 States	Stem: Fig. 3.6 states. A: New, Ready, Running, Blocked, Exit.
R3.4 Preempt	Stem: preempt a process. A: reclaim CPU before the process voluntarily yields or finishes.
R3.5 Swapping	Stem: swapping and purpose. A: move process memory to/from disk to free memory and improve CPU use.
R3.6 Two blocked	Stem: why Fig. 3.9b has two blocked states. A: resident Blocked vs swapped-out Blocked/Suspend.
R3.7 Suspended	Stem: four suspended-process traits. A: not executable now; may wait; agent-suspended; explicitly activated.
R3.8 OS tables	Stem: OS management tables. A: memory, I/O, file, process.
R3.9 PCB info	Stem: three PCB categories. A: identification, processor state, process control.
R3.10 Modes	Stem: why user/kernel modes. A: protect OS, PCBs, memory, I/O, privileged instructions.
R3.11 Create	Stem: OS steps to create process. A: PID, space, PCB, links, other tables.
R3.12 Int/trap	Stem: interrupt vs trap. A: external asynchronous event vs instruction-caused exception.
R3.13 Examples	Stem: three interrupts. A: clock, I/O, memory fault.
R3.14 Switches	Stem: mode switch vs process switch. A: privilege change vs changing the running process.

Chapter 3 Problem Stems

P3.1 Transitions	R/RUN/B/NR table: 1 dispatch, 2 preempt, 3 event wait, 4 event occurs, 5 ready swap-out, 6 blocked swap-out.
P3.2 Timeline	At $t = 22$: P1 B(d3 read), P3 B(d2 read), P7 B(d3 write), P5 likely running. At $t = 37$: P1/P3 ready, P5 Blocked/Suspend, P7 blocked, P8 running. At $t = 47$: P5 ready, P7 blocked, P8 exit; scheduler decides which ready process runs.
P3.3 Seven states	Possible direct moves: N→R/RS/E; R→Run/RS/E; Run→R/B/RS/E; B→R/BS/E; RS→R/E; BS→B/RS/E; all other direct moves are impossible in the standard model.
P3.4 Queues	Draw Ready queue, processor, per-event Blocked queues, Ready/Suspend queue, and per-event Blocked/Suspend queues.
P3.5 Policy	Balance priority and swap cost: swap in a Ready/Suspend process only when priority or aging benefit exceeds a threshold.
P3.6 VAX states	Many wait states give precise event/resource queues; memory-management or global resource waits need no resident/outswapped pair.
P3.7 Access modes	Kernel, Executive, Supervisor, User: finer protection and least privilege, but more complexity and switch overhead.
P3.8 Rings	Ring protection is nested; inner domains can access outer objects, so it cannot enforce selective need-to-know access well.

P3.9 Multi-wait	A process may wait for several events, e.g., network, keyboard, timeout; use wait descriptors or multiple queue links and remove stale links after wakeup.
P3.10 Fixed saves	Fixed interrupt save locations work only in simple nonnested systems; nested interrupts, multiprogramming, and process switches need PCB/kernel-stack context.
P3.11 Real-time	A nonpreemptive kernel gives unbounded latency, priority inversion, and missed deadlines; real-time systems need bounded response.
P3.12 Fork	Successful <code>fork()</code> prints two lines: 0 in the child and child PID in the parent; order is scheduler-dependent.

Chapter 3 Fast Recall

- Process = program in execution; PCB makes pause/resume possible.
- Ready lacks CPU; Blocked lacks an event; Suspended is out of main memory.
- Preemption takes CPU from a process before it yields.
- Interrupts are external; traps are instruction-caused; supervisor calls request OS services.
- A mode switch can stay in the same process; a process switch always changes processes.
- Context switching saves old PCB state and restores the selected process state.